

Министерство науки и высшего образования Российской
Федерации

Сыктывкарский лесной институт (филиал) федерального
государственного бюджетного образовательного учреждения
высшего образования «Санкт-Петербургский государственный
лесотехнический университет имени С. М. Кирова» (СЛИ)

**ИТОГОВЫЙ ОТЧЁТ
О ПРОХОЖДЕНИИ УЧЕБНОЙ ПРАКТИКИ**

CMS для парикмахерских и барбершопов «Сайтама»
Постановка задачи, архитектура и результаты разработки

Версия документа 1.0

00000000.230176.001.ОТ.01.1-М

Выполнил: Решетняк В. А., студент гр. 3356
подпись _____

Руководитель практики: Оганезова Н. А.
подпись _____

Начало практики: 22.06.2026

Конец практики: 18.07.2026

Сыктывкар 2026

Содержание

ВВЕДЕНИЕ	2
1 ПОСТАНОВКА ЗАДАЧИ	3
2 АВТОМАТИЗИРУЕМЫЕ БИЗНЕС-ПРОЦЕССЫ	4
3 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ И АРХИТЕКТУРНЫЕ ШАБЛОНЫ	6
3.1 Общая схема	6
3.2 Архитектурные шаблоны бэкенда	6
3.3 Архитектура фронтенда	7
4 ТРЕБОВАНИЯ ЗАКОНОДАТЕЛЬСТВА	9
5 ОБОСНОВАНИЕ ВЫБОРА СРЕДСТВ РЕАЛИЗАЦИИ	11
6 ДОПОЛНИТЕЛЬНЫЕ ИНСТРУМЕНТЫ РАЗРАБОТКИ	13
7 СТРУКТУРА БАЗЫ ДАННЫХ	15
7.1 Инфологическая модель	15
7.2 Даталогическая модель	16
8 ОПИСАНИЕ РАЗРАБОТАННОГО ПРИЛОЖЕНИЯ	19
8.1 Создание записи (бронирование)	19
8.2 Первый запуск системы (мастер /setup)	20
9 ПОДКЛЮЧЕНИЕ АНАЛИТИКИ	21
10 РАЗМЕЩЕНИЕ В СЕТИ ИНТЕРНЕТ	22
ЗАКЛЮЧЕНИЕ	23
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	25
ПРИЛОЖЕНИЕ А. Диаграммы последовательностей	27

ВВЕДЕНИЕ

Область применения документа. Документ подводит итог разработки CMS «Сайтама»: формулирует решаемую задачу, описывает автоматизируемые бизнес-процессы, архитектуру и применённые архитектурные шаблоны, требования законодательства и их реализацию, обоснование выбора технологического стека, состав вспомогательных инструментов разработки, структуру базы данных, работу ключевых сценариев приложения, интеграцию аналитики и способ размещения системы в сети Интернет.

Перечень документации, с которой связан отчёт. Три документа проекта детализируют то, что в настоящем отчёте описано на уровне обзора: техническое задание – первоначальную постановку задачи и полный перечень функциональных требований (карточки ФТ-001–ФТ-015); руководство пользователя – пошаговые инструкции по работе с системой для всех четырёх ролей (клиент, мастер, администратор точки, владелец сети); руководство администратора – развёртывание, эксплуатацию и резервное копирование серверной части. Настоящий отчёт этот материал не дублирует, а ссылается на него там, где нужны подробности.

Уровень подготовки читателя. Документ рассчитан на читателя с базовым представлением о веб-разработке: клиент-серверная архитектура, REST API, реляционные базы данных и SQL, основы контейнеризации (Docker). Специальных знаний Python/FastAPI или Vue.js для понимания отчёта не требуется – технологические решения объясняются по мере изложения.

1 ПОСТАНОВКА ЗАДАЧИ

Исходная проблема. Барбершоп «Сайтама», выбранный пилотным объектом внедрения, вёл запись клиентов и учёт услуг вручную (телефон, бумажный журнал, разрозненные записи) – это порождало типовые для отрасли проблемы: двойные записи одного мастера на одно и то же время, отсутствие напоминаний клиентам и, как следствие, неявки (no-show), отсутствие единого каталога услуг с ценами, различающимися по мастеру, и отсутствие у владельца оперативной картины по выручке и загруженности без ручного подсчёта.

Задача. Разработать веб-приложение, автоматизирующее полный цикл операционной деятельности барбершопа: онлайн-запись клиента без участия администратора, ведение каталога услуг с индивидуальным прайсом по мастерам, автоматические напоминания о визите, формирование отчётов по выручке и загруженности, и – что не было предусмотрено первоначальной редакцией технического задания, но было добавлено по ходу практики – поддержку сети из нескольких точек с ролью владельца сети поверх администратора отдельной точки.

Измеримые цели (полный перечень с формулировками показателей – техническое задание, раздел 2.2): сокращение времени оформления записи клиентом до 2 минут; исключение двойных бронирований техническими средствами (ограничением целостности СУБД, а не организационно); снижение доли неявок за счёт автоматических напоминаний; поддержка индивидуального прайса услуги по мастеру; отказ от бумажных/табличных журналов в пользу единого хранилища данных; формирование отчёта по выручке и загруженности без участия разработчика.

Границы задачи. Вне рамок разработки остались: приём онлайн-оплаты (оплата очно, см. техническое задание, раздел 3.2), бухгалтерский и складской учёт, CRM-функции сверх базового профиля клиента (сегментация, маркетинговые рассылки, программы лояльности). Эти ограничения не изменились по ходу практики.

2 АВТОМАТИЗИРУЕМЫЕ БИЗНЕС-ПРОЦЕССЫ

Система автоматизирует пять бизнес-процессов, выделенных при обследовании предметной области (техническое задание, раздел 2.1):

- **Онлайн-запись клиента** (Зап-1) – выбор услуги, мастера, свободного времени и подтверждение записи без участия администратора; для незарегистрированного клиента запись создаёт учётную запись автоматически (см. раздел 8).
- **Управление услугами и прайсом** (Усл-2) – единый каталог услуг с ценой, индивидуальной по каждой паре «мастер – услуга».
- **Уведомления и напоминания** (Увед-3) – автоматическая отправка напоминаний за 24 и за 2 часа до визита.
- **Формирование отчётов** (Отчёт-4) – выручка, загруженность мастеров, топ-услуги за произвольный период, без привлечения разработчика (раздел 9.3).
- **Управление контентом сайта** – редактирование главной страницы, страницы мастеров и настроек точки без правки кода.

Расхождение с первоначальным планом: канал уведомлений. Техническое задание закладывало многоканальную доставку уведомлений (email, SMS, Telegram Bot API) с выбором канала по приоритету получателя. По факту реализован только канал email (SMTP) – Telegram Bot API и SMS-провайдер не подключались; в первой миграции схемы БД была предусмотрена таблица для многоканального журнала уведомлений, но она не была задействована ни одним сервисом и впоследствии удалена как мёртвая схема (раздел 7). Причина сужения – типовой для практики ком-промисс «время на практику ограничено, email закрывает и подтверждение регистрации, и восстановление пароля, и напоминания одним и тем же уже написанным механизмом отправки»: расширение до Telegram/SMS осталось в объёме задокументированных, но не реализованных возможностей (см. заключение).

Процесс, добавленный сверх первоначального плана. По ходу практики к пилотному барбершопу «Сайтама» добавилась поддержка сети из нескольких точек: роль «владелец сети» (шире «администратора точки»), переключение между точками, назначение администратора конкретной точ-

ке. Функции по ролям (клиент/мастер/администратор/владелец) с полным перечнем операций – руководство пользователя, разделы 3.4.1–3.4.4.

3 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ И АРХИТЕКТУРНЫЕ ШАБЛОНЫ

3.1 Общая схема

Система – классическое клиент-серверное веб-приложение: одностраничное приложение (SPA) на Vue 3, обращающееся к REST API на FastAPI, который работает с единственной базой данных PostgreSQL. Рисунок 1 показывает слоистую архитектуру бэкенда и то, куда в неё встраивается фронтенд; топология развёртывания (Docker-контейнеры, реверс-прокси, TLS) – предмет отдельного документа (руководство администратора, раздел 3.1) и здесь не повторяется.

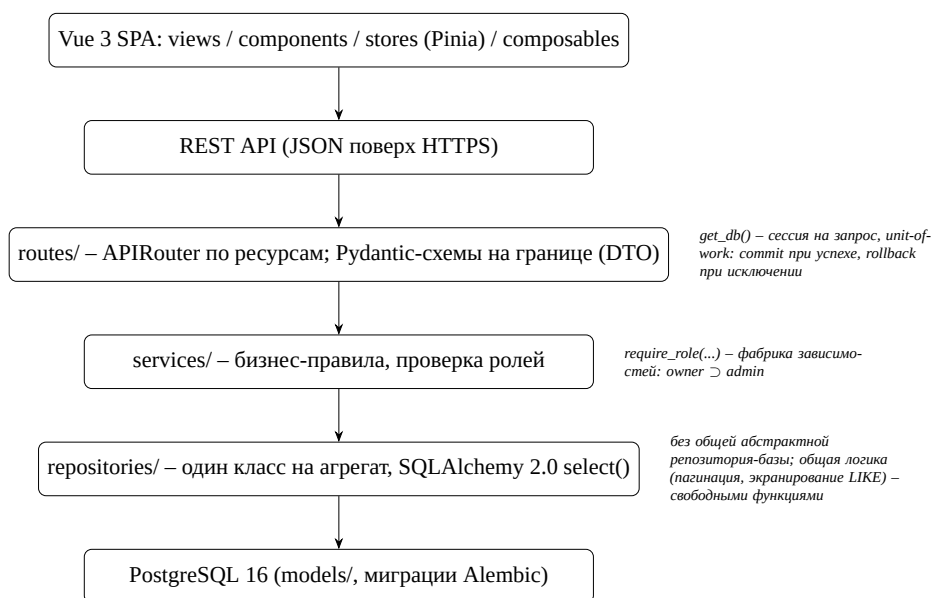


Рисунок 1 – Слоистая архитектура: от фронтенда до базы данных

3.2 Архитектурные шаблоны бэкенда

Backend (backend/app/) построен как строгая однонаправленная слоистая архитектура – routes $\rightarrow$ services $\rightarrow$ repositories $\rightarrow$ models, – с параллельным пакетом schemas/ Pydantic-схем, используемых только на границе запроса: сервисы всегда возвращают `XResponse.model_validate(orm_obj)`, поэтому «сырые» ORM-объекты никогда не попадают наружу через HTTP. Внутри этой схемы использовано несколько именованных шаблонов проектирования:

- **DTO на границе (Data Transfer Object).** Разделение schemas/ (что видит клиент API) и models/ (как данные лежат в БД) – изменение внутреннего представления не ломает контракт API, и наоборот.
- **Repository.** Один класс-репозиторий на агрегат (UserRepository, AppointmentRepository, SalonRepository и т. д.) – вся работа с SQLAlchemy сосредоточена в этом слое; сервисы не составляют SQL-запросы напрямую. Общей абстрактной репозитория-базы или generic CRUD-класса нет осознанно – общая логика (пагинация, экранирование LIKE) вынесена свободными функциями, а не наследованием, что проще для относительно небольшого числа репозиториях проекта.
- **Dependency Injection через Depends.** Отдельного DI-контейнера нет – вся инъекция зависимостей идёт штатным механизмом FastAPI. get_db() – сессия БД на один запрос, реализующая шаблон **Unit of Work**: один commit при успешном завершении запроса, rollback при любом исключении, db.close() в finally.
- **Dependency-factory.** require_role(*roles) – функция, возвращающая функцию-зависимость; так собраны get_current_client/master/admin/owner. Поскольку owner $\supset$ admin по бизнес-правилу, get_current_admin проверяет обе роли сразу.
- **Явное разрешение области видимости (salon scoping).** Хелпер resolve_salon_scope(current_user, salon_id) – не Depends, а обычная функция, вызываемая внутри уже защищённых ролей эндпоинтов: для owner отсутствие salon_id означает «вся сеть», для admin – доступ жёстко ограничен собственной точкой (несовпадение – код ответа 403).

3.3 Архитектура фронтенда

Frontend (frontend/src/) – Vue 3 на Composition API (<script setup> повсеместно, без TypeScript), Vue Router 4, управление состоянием – Pinia (шесть «setup-хранилищ» – auth, salon, setup, siteContent, masterProfile, toast); Vuex в проекте не используется. Небольшая переиспользуемая логика (состояние ошибок формы, отложенное наблюдение) вынесена в composables/, а не в хранилища. Маршрутизация – один файл router/index.js с ленивой загрузкой всех маршрутов и единым глобальным beforeEach-хуком, который, помимо

проверки ролей (meta.roles), принудительно блокирует всё приложение мастером первого запуска /setup, пока тот не завершён (см. раздел 8.2).

Два решения на уровне фронтенда стоит выделить отдельно как архитектурные, не просто UI-детали:

- **Токен сессии – только в httpOnly cookie**, никогда не в localStorage/JS-переменной (снижает риск кражи токена через XSS). Обновление токена – через axios-интерцептор с *single-flight*: при 401 запускается один POST/auth/refresh, параллельные запросы дожидаются общего promise вместо каждый – своего обновления, и только при неудаче самого refresh срабатывает глобальный logout.

- **Тема оформления как настройка, а не константа.** Цвета бренда (brand/accent/ink/stone) объявлены в Tailwind не фиксированными значениями, а CSS custom properties, переключаемыми во время выполнения из настроек сайта, заданных администратором – один и тот же собранный фронтенд обслуживает точки с разным оформлением без пересборки.

4 ТРЕБОВАНИЯ ЗАКОНОДАТЕЛЬСТВА

Согласно техническому заданию (раздел 1.4), система должна соответствовать Федеральному закону № 152-ФЗ «О персональных данных» и Федеральному закону № 149-ФЗ «Об информации, информационных технологиях и о защите информации». Ниже – не повтор требований, а то, что из них фактически реализовано в коде к моменту завершения практики.

- **Согласие на обработку персональных данных.** Форма регистрации требует явной отметки согласия (чекбокс со ссылкой на политику конфиденциальности) – отправка формы блокируется, пока согласие не отмечено; то же условие распространено на вход через VK ID (кнопка входа через VK неактивна без отметки согласия).

- **Страница политики конфиденциальности** (маршрут /privacy, ссылка – в подвале сайта и на форме регистрации) – десять разделов, соответствующих типовой структуре документа по 152-ФЗ (правовые основания, категории данных, цели обработки, права пользователя, раскрытие использования cookie и аналитики, контактные данные оператора и т. д.).

- **Честно задокументированное ограничение.** Раздел реквизитов оператора персональных данных на странице политики сам себя помечает как незаполненный шаблон (плашка «сайт в разработке», поля наименования/ИНН/ОГРН – заглушки) и прямо указывает, что документ не имеет юридической силы по 152-ФЗ до заполнения реальными реквизитами. Это осознанное решение, а не недосмотр: механизм (форма согласия, страница политики, раскрытие использования аналитики) реализован полностью, реальные юридические реквизиты появятся при регистрации барбершопа как оператора персональных данных вне рамок учебной практики.

- **Раскрытие использования cookie и аналитики.** Отдельного баннера согласия на cookie нет – используется формулировка «продолжение использования сайта означает согласие», раскрытая в §8 политики конфиденциальности вместе с фактом использования Яндекс.Метрики (раздел 9.2).

- **Хранение и удаление персональных данных.** Пароли не хранятся в открытом виде (хэш пароля, поле password_hash); удаление учётной записи при наличии связанных записей о визитах технически заблокирова-

но ограничением ссылочной целостности (ON DELETE RESTRICT) – вместо удаления применяется мягкая блокировка (is_blocked), что само по себе требование к порядку последующей ручной обработки запросов на удаление данных вне объёма настоящей практики.

5 ОБОСНОВАНИЕ ВЫБОРА СРЕДСТВ РЕАЛИЗАЦИИ

Таблица 1 – Технологический стек и обоснование выбора

Средство	Обоснование
FastAPI (Python)	Асинхронный фреймворк с валидацией запросов через Pydantic «из коробки» и автогенерацией интерактивной документации API (OpenAPI/Swagger, /api/docs) без отдельного описания контракта вручную.
PostgreSQL 16	Единственная СУБД, рассмотренная всерьёз, – из-за ограничения целостности EXCLUDE USING gist: требование «0 двойных бронирований» (техническое задание, раздел 2.2) решено <i>на уровне схемы БД</i> , а не проверкой в коде приложения перед записью, которая при параллельных запросах подвержена состоянию гонки.
SQLAlchemy 2.0 + Alembic	Типизированный ORM (Mapped[...]) и версионизируемые миграции схемы вместо ручных SQL-скриптов – критично при 15 последовательных миграциях за время практики, включая нетривиальные (расширение enum-типа отдельной транзакцией, раздел 7).
Pydantic	Валидация входных данных и сериализация ответов единым механизмом, общим с автогенерацией документации FastAPI.
Vue 3 (Composition API) + Pinia	Компактный реактивный фронтенд без избыточного бойлерплейта Vuex/Redux – оправдано масштабом проекта (один фронтенд-разработчик, шесть хранилищ состояния).

Продолжение таблицы 1

Средство	Обоснование
Tailwind CSS	Единая цветовая и типографическая система (требование к эргономике, техническое задание, раздел 4.1.5) без написания собственного CSS-фреймворка с нуля.
Docker Compose + Caddy	Воспроизводимое развёртывание и автоматический TLS-сертификат Let's Encrypt без ручной настройки nginx/certbot; подробности – руководство администратора, раздел 3.1.

Осознанно упрощено против первоначального плана: без Celery и Redis. Техническое задание закладывало очередь фоновых задач (Celery + Redis) для рассылки напоминаний. По факту рассылка реализована проще – фоновым `asyncio`-циклом внутри самого процесса бэкенда (раздел 7.3 руководства администратора: планировщик напоминаний работает «внутри процесса бэкенда, без внешней очереди задач»). Решение сознательное, а не недоделка: система в текущей версии рассчитана ровно на один работающий экземпляр контейнера `backend` (без горизонтального масштабирования, руководство администратора, раздел 3.2), и при этом ограничении отдельная очередь задач с брокером сообщений добавляла бы эксплуатационную сложность (ещё один сервис, ещё одна точка отказа) без практической выгоды. Тот же аргумент применён к защите от перебора паролей и ограничению частоты запросов – оба реализованы в памяти процесса (“`slowapi`“ с `Limiter` по IP и таблица `login_attempts` соответственно), без Redis.

6 ДОПОЛНИТЕЛЬНЫЕ ИНСТРУМЕНТЫ РАЗРАБОТКИ

Таблица 2 – Инструменты разработки, тестирования и контроля качества

Инструмент	Назначение
ruff	Линтер Python (E,F,B,UP,SIM,C4); автоформатирование (ruff format) осознанно не подключено – переформатировало бы весь проект, ломая уже расставленное ручное выравнивание.
ESLint 9 + eslint-plugin-vue	Линтер JS/Vue (flat config, vue/flat/essential); Prettier настроен (npm run format) и доступен локально, но в CI не проверяется – сознательный выбор в пользу линтинга без навязывания единого форматирования.
Pyright	Проверка типов Python в режиме basic (pyrightconfig.json в корне репозитория) – только локально, на CI шага проверки типов нет.
pytest (unit + integration)	Модульные тесты (backend/tests, без реальной БД, сервисы получают тестовые репозитории) и отдельно помеченные интеграционные тесты (backend/tests/integration, запускаются только явно, -m integration) – на настоящем Postgres через httpx.AsyncClient, покрывают то, что структурно недоступно модульным тестам: ограничение EXCLUDE, уникальность slug точки, area видимости по точкам сети, полный сценарий записи «от начала до конца».
Vitest + @vue/test-utils	Модульные тесты фронтенда (хранилища, composables).

Продолжение таблицы 2

Инструмент	Назначение
Playwright + @axe-core/playwright	End-to-end тесты в браузере (Chromium) и автоматическая регрессия по доступности (accessibility) на этой же тестовой инфраструктуре.
GitHub Actions (CI)	Восемь параллельных job'ов на каждый push/PR в main: тесты и линт бэкенда, интеграционные тесты бэкенда на реальном Postgres, сборка и линт фронтенда, unit- и e2e-тесты фронтенда, синтаксическая проверка docker-compose.yml. Автоматического деплоя по CI нет – обновление прод-версии выполняется вручную (scripts/deploy.sh, руководство администратора, раздел 3.4.1).
pip-audit / npm audit	Проверка зависимостей на известные уязвимости как отдельные шаги CI.
Dependabot	Еженедельные автоматические PR на обновление зависимостей (pip, npm, Docker-образы обоих Dockerfile, GitHub Actions).

7 СТРУКТУРА БАЗЫ ДАННЫХ

7.1 Инфологическая модель

Модель разделена на две схемы для читаемости: рисунок 2 – сущности бизнес-домена (точки, пользователи, мастера, услуги, записи, отзывы); рисунок 3 – сущности, обеспечивающие безопасность входа (сессии, токены, попытки входа), и настройки сайта. Даталогическая модель (таблица 3) описывает все эти сущности на уровне столбцов и ограничений БД.

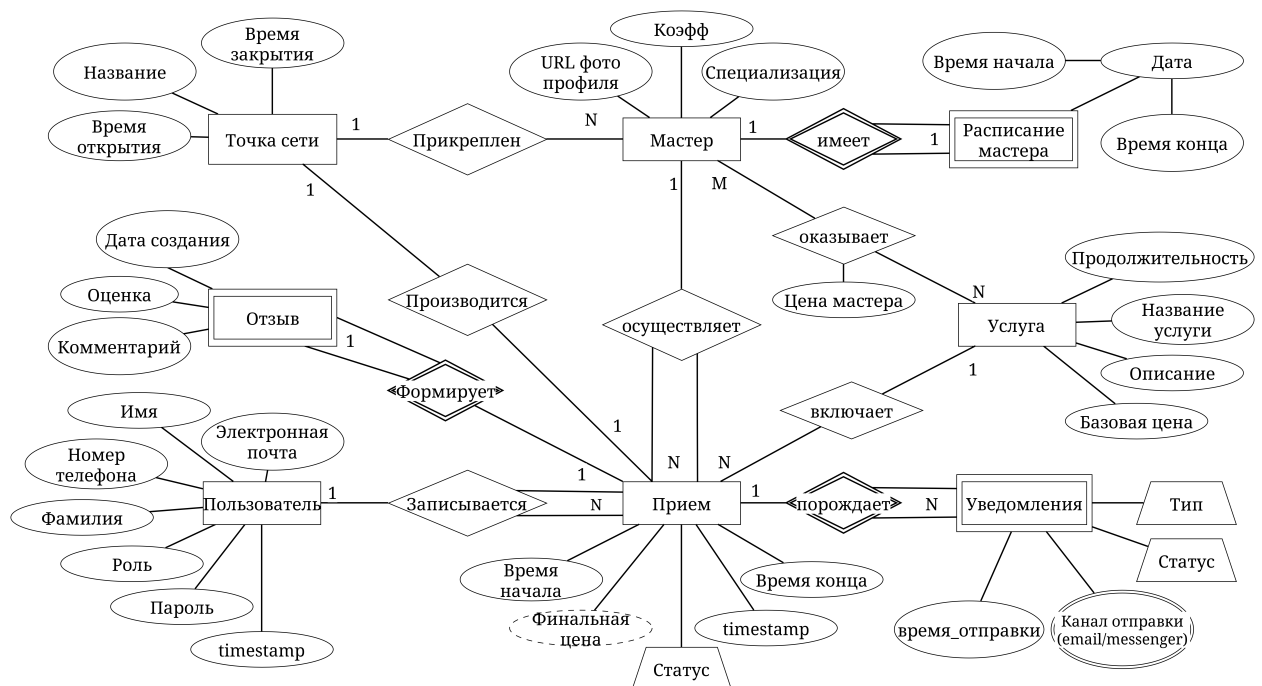


Рисунок 2 – Инфологическая модель – бизнес-домен (актуальная схема БД)

Отличия от инфологической модели технического задания. Модель технического задания (раздел 5.1.2) обновлена вместе с этим отчётом и включает сущность «Точка сети» – многоточечность (раздел 2) отражена в обоих документах одинаково. Оставшиеся отличия – на уровне деталей, не затронутых обновлением диаграммы. Роль owner не выделена отдельной сущностью, а добавлена как четвёртое значение перечисления role у существующей сущности «Пользователь»: owner $\supset$ admin проверяется в коде сервисного слоя (раздел 3), а не отдельной таблицей. Сущность «Журнал аудита» (audit_log) из технического задания не реализована – её задачу частично закрывает таблица попыток входа (login_attempts, обязательная по требованию 5.1 технического задания к логированию попыток авторизации), но универсального

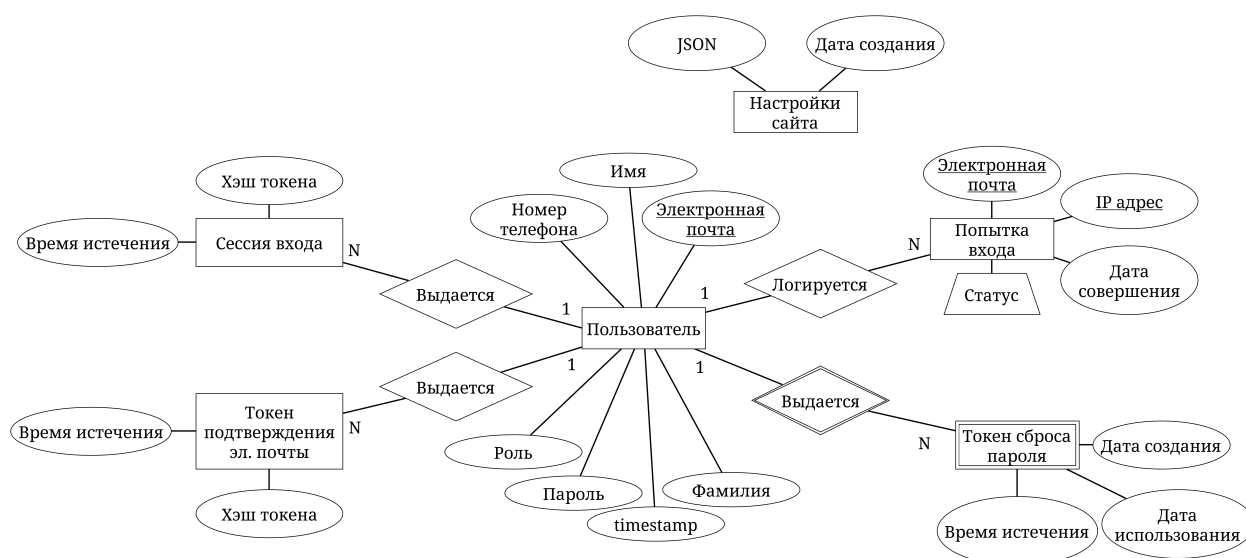


Рисунок 3 – Инфологическая модель – аутентификация и настройки сайта

журнала изменений критичных операций в системе нет (см. заключение).

7.2 Даталогическая модель

Таблица 3 перечисляет все 13 таблиц фактической схемы БД (15 миграций Alembic, backend/alembic/versions/0001_initial.py – 0015_salon_scoping.py) с ключевыми столбцами и ограничениями.

Таблица 3 – Таблицы базы данных

Таблица	Ключевые столбцы и ограничения
users	id, role (client/master/admin/owner), salon_id (FK, обязателен только для admin – CHECK), email/password_hash (оба допускают NULL – аккаунты только через VK ID), vk_user_id (частичный уникальный индекс), is_blocked, token_version.
salons	id, name, slug (уникальный), address, open_time/close_time, is_active (мягкое закрытие – ON DELETE RESTRICT со стороны masters/appointments).

Продолжение таблицы 3

Таблица	Ключевые столбцы и ограничения
masters	id, user_id (FK), salon_id (FK, NOT NULL), specialization, photo_url, coefficient (множитель к базовой цене услуги, по умолчанию 1.00), is_active. Рейтинг мастера отдельным столбцом не хранится – вычисляется по связанным reviews.
schedules	id, master_id (FK), day_of_week (0–6), start_time/end_time, is_working.
services	id, name, description, price, duration_min, is_active – без прямой привязки к салону; принадлежность точке определяется через мастера в master_services.
master_services	Композитный первичный ключ (master_id, service_id) – без отдельного id; price_override (NULL $\Rightarrow$ итоговая цена = services.price $\times$ masters.coefficient).
appointments	id, client_id (FK), master_id (FK), service_id (FK), salon_id (снимок точки мастера на момент создания записи), start_time/end_time, final_price (зафиксирована на момент создания, не меняется при изменении прайса), status, reminder_24h_sent_at/reminder_2h_sent_at. Ограничение EXCLUDE USING gist по пересечению временного диапазона одного мастера – защита от двойного бронирования на уровне СУБД.
reviews	id, appointment_id (FK, уникальный), client_id/master_id/service_id (денормализованы из записи на момент создания отзыва), rating, comment, is_published.

Продолжение таблицы 3

Таблица	Ключевые столбцы и ограничения
sessions	id, user_id (FK), token_hash (SHA-256) – серверное хранилище refresh-сессий (в техническом задании не выделялось отдельной сущностью).
password_reset_tokens	id, user_id (FK), token, expires_at.
email_verification_tokens	id, user_id (FK), token, expires_at.
login_attempts	id, признак успеха, метка времени – основа блокировки входа после серии неудачных попыток (раздел 4) и частичная замена журнала аудита (см. выше).
site_settings	Единственная строка, content (JSONB) – настройки сайта; в техническом задании описывалась плоскими столбцами, по факту хранится одним JSON-документом.

Удалено как мёртвая схема. Первая миграция создавала таблицы notification_log/notifications под многоканальную рассылку из технического задания – ни один сервис или маршрут никогда не писал в них, воркера для чтения тоже не было; таблица удалена отдельной миграцией (0005_drop_notifications.py) как вводящая в заблуждение мёртвая схема, а не как потеря функциональности.

8 ОПИСАНИЕ РАЗРАБОТАННОГО ПРИЛОЖЕНИЯ

Полный перечень операций по каждой из четырёх ролей – руководство пользователя, разделы 3.4.1–3.4.4. Здесь – сквозные технические сценарии двух ключевых процессов на уровне «фронтенд → бэкенд → БД».

8.1 Создание записи (бронирование)

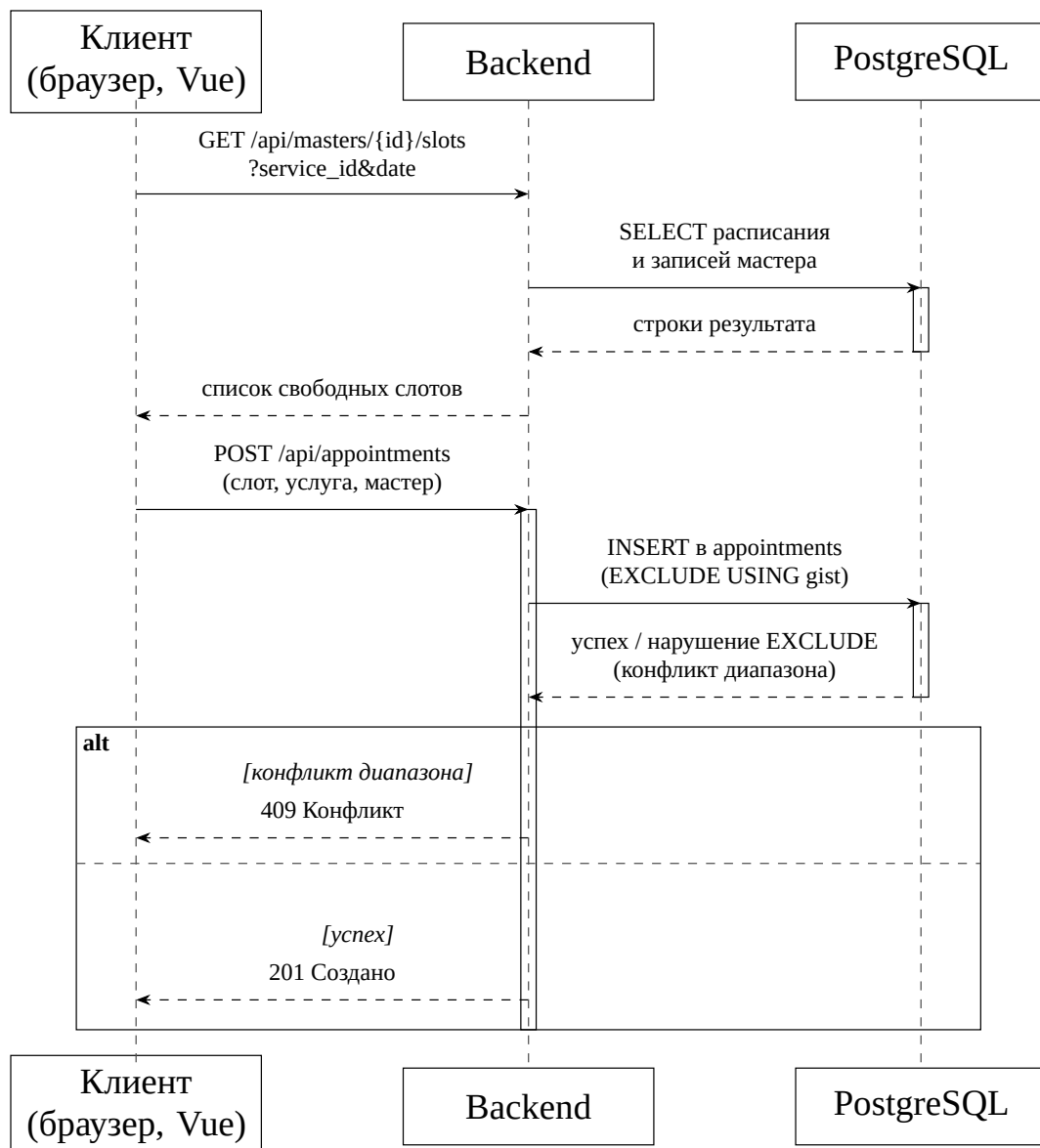


Рисунок 4 – Диаграмма последовательности: создание записи

Повторная проверка доступности слота на шаге 6 на уровне СУБД, а не только на уровне кода перед вставкой, – прямая реализация цели «0 инцидентов двойного бронирования» из технического задания (раздел 2.2): при

конкурентной попытке два запроса могут одновременно пройти проверку на шаге 1–4, но лишь один пройдёт INSERT на шаге 6, второй получит 409 от самой базы данных.

8.2 Первый запуск системы (мастер /setup)

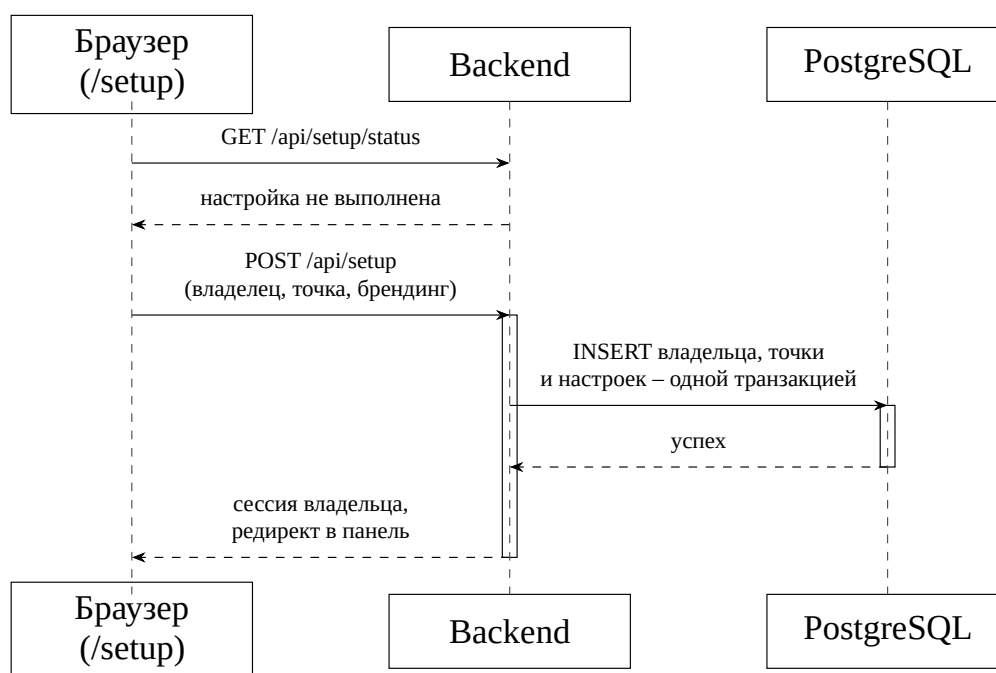


Рисунок 5 – Диаграмма последовательности: первый запуск (/setup)

Маршрут /setup проверяется единым глобальным `router.beforeEach`-хуком фронтенда перед *каждой* навигацией (раздел 3), поэтому попасть в остальную часть приложения, минуя этот сценарий, нельзя; после успешного шага 6 статус настройки самоблокируется навсегда (руководство администратора, раздел 3.3.4).

9 ПОДКЛЮЧЕНИЕ АНАЛИТИКИ

В системе три независимых источника аналитики, которые не стоит путать между собой:

- **Веб-аналитика (Яндекс.Метрика) – подключена и активна.** Счётчик загружается отдельным статическим файлом (/metrika.js), а не инлайн-скриптом в index.html, – сознательное решение ради политики безопасности контента (CSP): script-src остаётся 'self' без исключения 'unsafe-inline'. Включены webvisor, clickmap, отслеживание переходов и точный подсчёт отказов. Использование раскрыто в §8 политики конфиденциальности (раздел 5).

- **Мониторинг ошибок (Sentry) – код готов, отключён по умолчанию.** Инициализация присутствует и на бэкенде, и на фронтенде, но выполняется только при заданном SENTRY_DSN/VITE_SENTRY_DSN – до появления учётной записи Sentry (или self-hosted-аналога) обе интеграции неактивны. Порядок подключения – руководство администратора, раздел 3.4.3.

- **Бизнес-аналитика – собственные отчёты по данным приложения.** Отдельно от двух пунктов выше: раздел «Статистика» (KPI-плитки, график регистраций за 30 дней) и раздел «Отчёты» (выручка и загруженность по дням/мастерам/услугам с экспортом в Excel) – оба на данных из собственной БД проекта, без внешнего сервиса. Реализуют цель технического задания «формирование отчёта по выручке и загруженности без привлечения разработчика» (раздел 2.2); полное описание экранов – руководство пользователя, раздел 3.4.3.

10 РАЗМЕЩЕНИЕ В СЕТИ ИНТЕРНЕТ

Полный порядок развёртывания, включая перечень необходимых портов, подключение домена и TLS, первичную настройку и восстановление после сбоя, – предмет отдельного документа (руководство администратора, разделы 3.1–3.5) и здесь не повторяется. Кратко: система разворачивается как согласованный набор Docker-контейнеров (база данных, backend, frontend, реверс-прокси Caddy с автоматическим TLS-сертификатом Let's Encrypt, ежедневное резервное копирование), наружу из этой топологии открыты только порты 80/443 контейнера Caddy, обновление выполняется вручную одной командой (`scripts/deploy.sh`) без автоматического деплоя по CI (раздел 6). На момент завершения практики система готова к развёртыванию на произвольном сервере с Linux и Docker по этой процедуре; сквозной пример – от пустого сервера до работающего сайта с доменом – руководство администратора, приложение Г.

ЗАКЛЮЧЕНИЕ

За время учебной практики (22.06.2026–10.08.2026) разработана и доведена до готовности к развёртыванию CMS «Сайтама» – веб-приложение для автоматизации онлайн-записи и операционной деятельности сети барбершопов.

Соответствие целям технического задания. Ключевые измеримые цели (раздел 2.2 технического задания) обеспечены технически: двойное бронирование исключено ограничением целостности СУБД, а не организационно (раздел 8); индивидуальный прайс по мастеру, единое хранилище данных и отчёт по выручке/загруженности без участия разработчика – реализованы и доступны в интерфейсе администратора (раздел 9.3). Часть целей (снижение доли неявок за счёт напоминаний, доля клиентов, выбирающих вход через VK ID) сформулирована как показатель, измеримый только на реальной эксплуатации, – их фактическое достижение не может быть подтверждено на этапе практики и требует данных работающей системы после ввода в реальную эксплуатацию.

Осознанно суженный или отложенный объём. Три пункта первоначального технического задания не реализованы – и в каждом случае это задокументированное решение, а не забытая работа: второй OAuth-провайдер MAX ID (реализован только VK ID, раздел 3); каналы уведомлений Telegram Bot API и SMS (реализован только email, раздел 2); отдельная таблица журнала аудита критичных операций `audit_log` (частично закрыта таблицей попыток входа `login_attempts`, раздел 7). Отдельно, очередь фоновых задач Celery + Redis из технического задания сознательно заменена более простым внутрипроцессным `asycio`-планировщиком (раздел 5) – здесь не сужение функциональности, а инженерное решение в пользу меньшей эксплуатационной сложности при заданном ограничении «один экземпляр backend».

Направления дальнейшего развития. Второй OAuth-провайдер и дополнительные каналы уведомлений можно добавить без изменения архитектуры (раздел 3 уже отделяет бизнес-логику сервисов от конкретного канала доставки/провайдера); горизонтальное масштабирование backend потребует пересмотра внутрипроцессных механизмов защиты от перебора паролей и ограничения частоты запросов (раздел 5) в пользу общего хранилища состо-

яния; полноценный журнал аудита – отдельная таблица с записью каждой критичной операции, а не только попыток входа.

Итоговое состояние проекта, полный перечень функциональных требований и инструкции по эксплуатации – см. соответственно техническое задание, руководство пользователя и руководство администратора.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

- 1) ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. – М.: Стандартинформ, 2020.
- 2) ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. – М.: Изд-во стандартов, 1978.
- 3) ГОСТ 19.105-78. Единая система программной документации. Общие требования к программным документам. – М.: Изд-во стандартов, 1978.
- 4) ГОСТ Р ИСО/МЭК 15408-3-2008. Информационная технология. Методы и средства обеспечения безопасности. Критерии оценки безопасности информационных технологий. Часть 3. – М.: Стандартинформ, 2008.
- 5) Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных».
- 6) Федеральный закон от 27.07.2006 № 149-ФЗ «Об информации, информационных технологиях и о защите информации».
- 7) FastAPI – официальная документация [Электронный ресурс]. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 09.08.2026).
- 8) SQLAlchemy 2.0 – официальная документация [Электронный ресурс]. – URL: <https://docs.sqlalchemy.org/en/20/> (дата обращения: 09.08.2026).
- 9) Alembic – официальная документация [Электронный ресурс]. – URL: <https://alembic.sqlalchemy.org/> (дата обращения: 09.08.2026).
- 10) Pydantic – официальная документация [Электронный ресурс]. – URL: <https://docs.pydantic.dev/> (дата обращения: 09.08.2026).
- 11) Vue.js – официальная документация [Электронный ресурс]. – URL: <https://vuejs.org/> (дата обращения: 09.08.2026).
- 12) PostgreSQL 16 – официальная документация [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/> (дата обращения: 09.08.2026).
- 13) Caddy – официальная документация [Электронный ресурс]. – URL: <https://caddyserver.com/docs/> (дата обращения: 09.08.2026).
- 14) Docker Compose – официальная документация [Электронный ре-

сурс]. – URL: <https://docs.docker.com/compose/> (дата обращения: 09.08.2026).

ПРИЛОЖЕНИЕ А. Диаграммы последовательностей

Диаграммы А.1–А.5 иллюстрируют порядок взаимодействия участников для ключевых бизнес-процессов системы, детализированных в карточках функциональных требований технического задания (раздел 4.2).

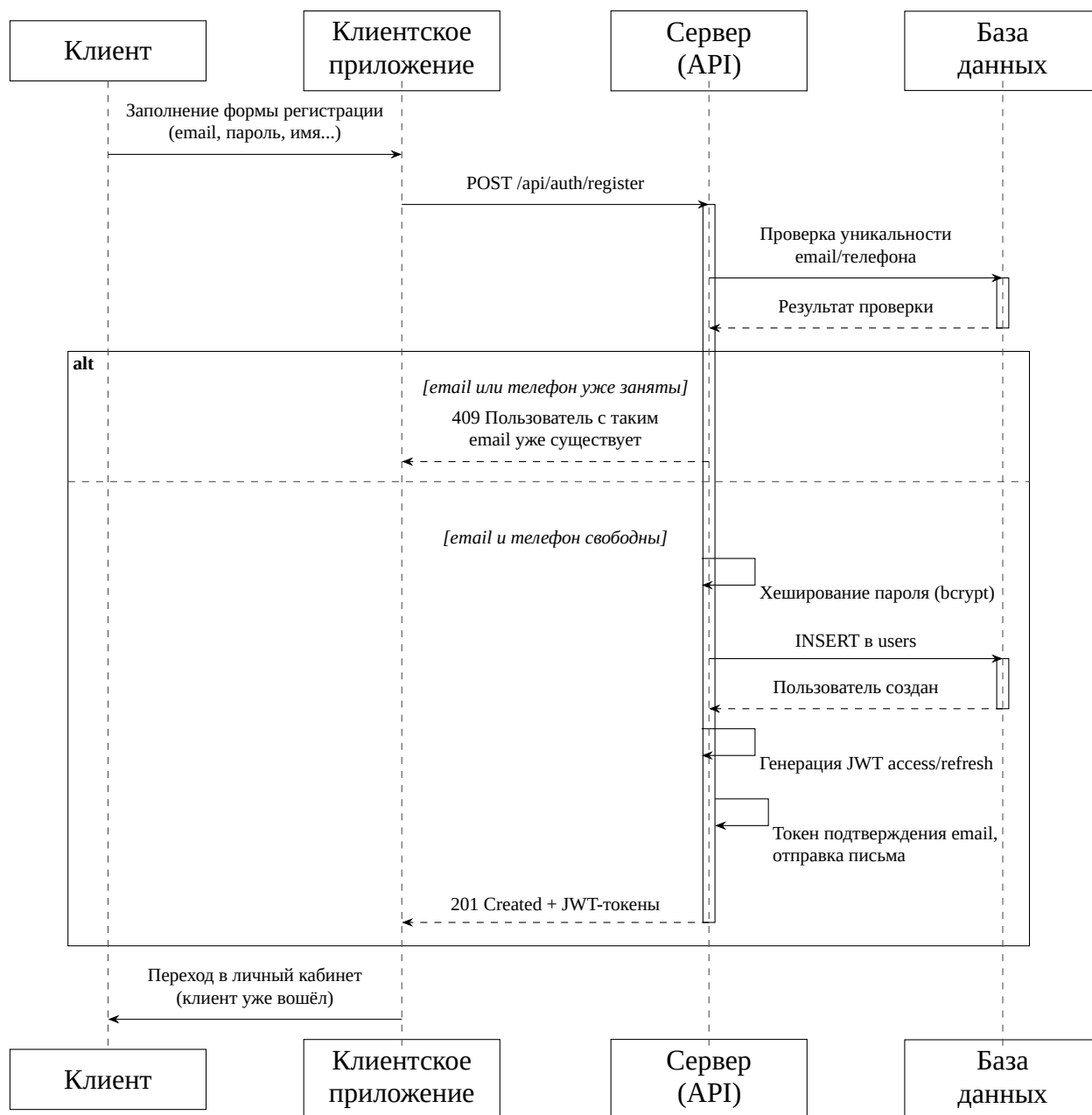


Рисунок А.1 – Диаграмма последовательностей бизнес-процесса «Регистрация» (FR-001)

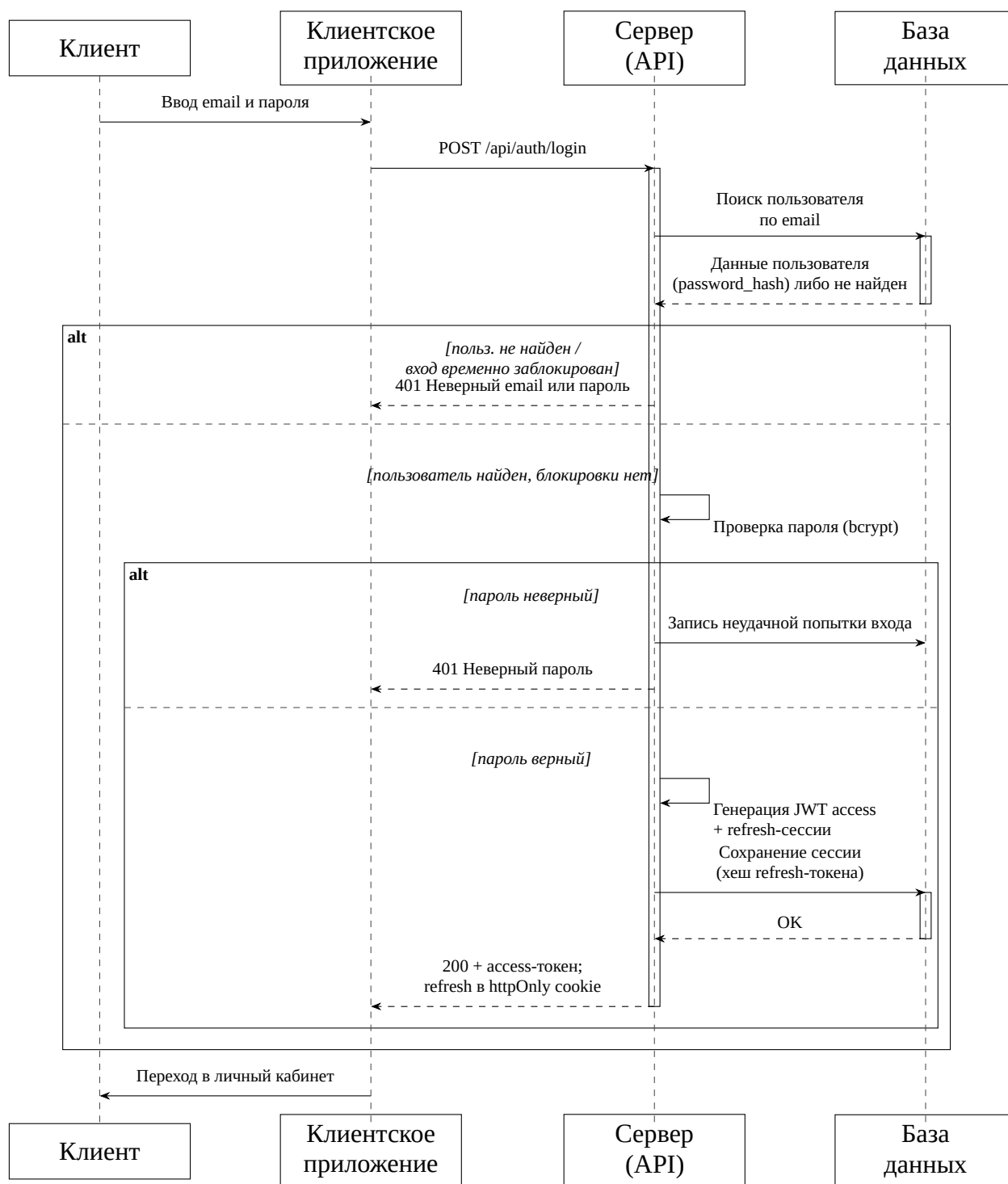


Рисунок А.2 – Диаграмма последовательностей бизнес-процесса «Вход по email и паролю»

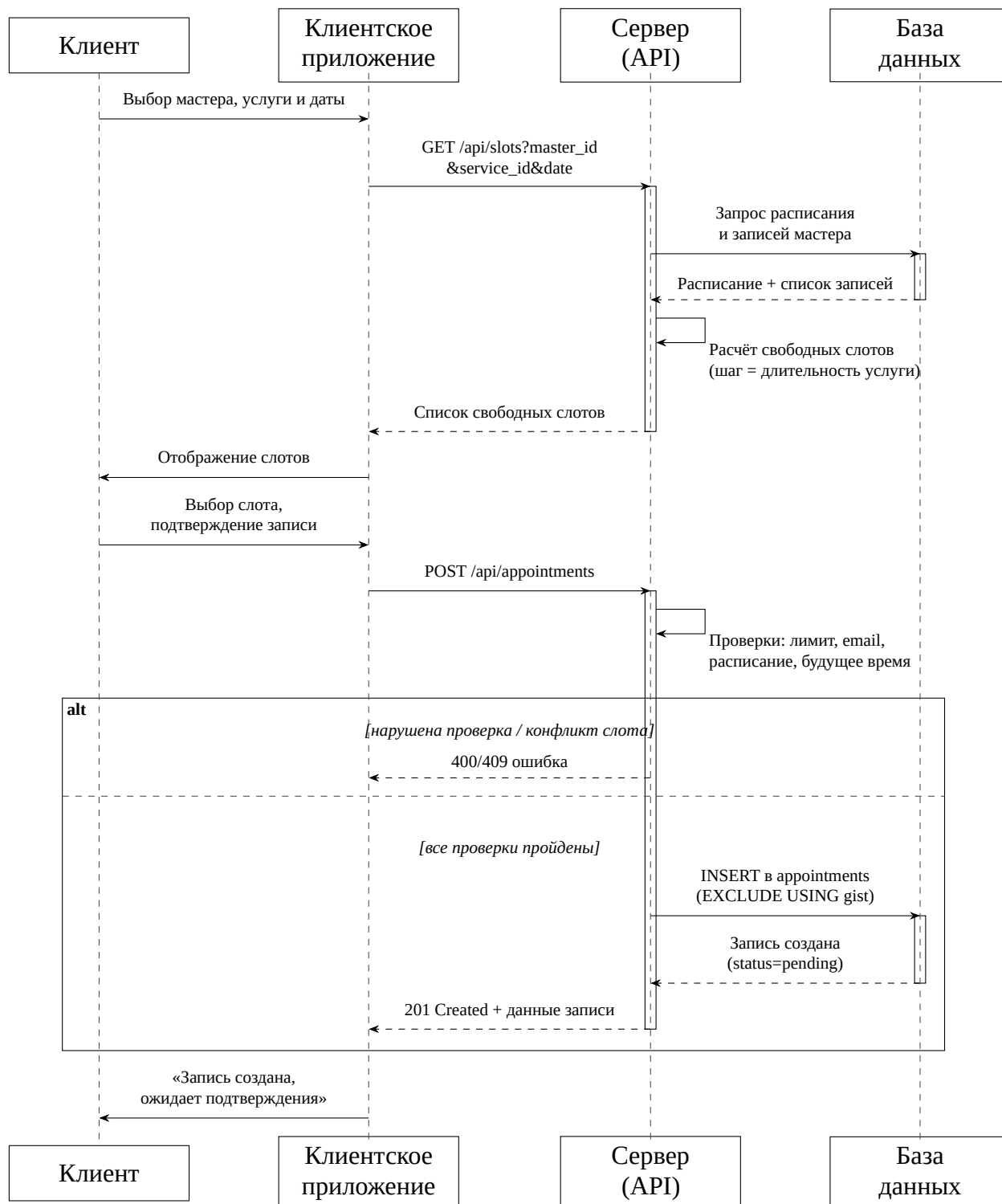


Рисунок А.3 – Диаграмма последовательностей бизнес-процесса «Онлайн-запись на приём» (FR-004, FR-005)

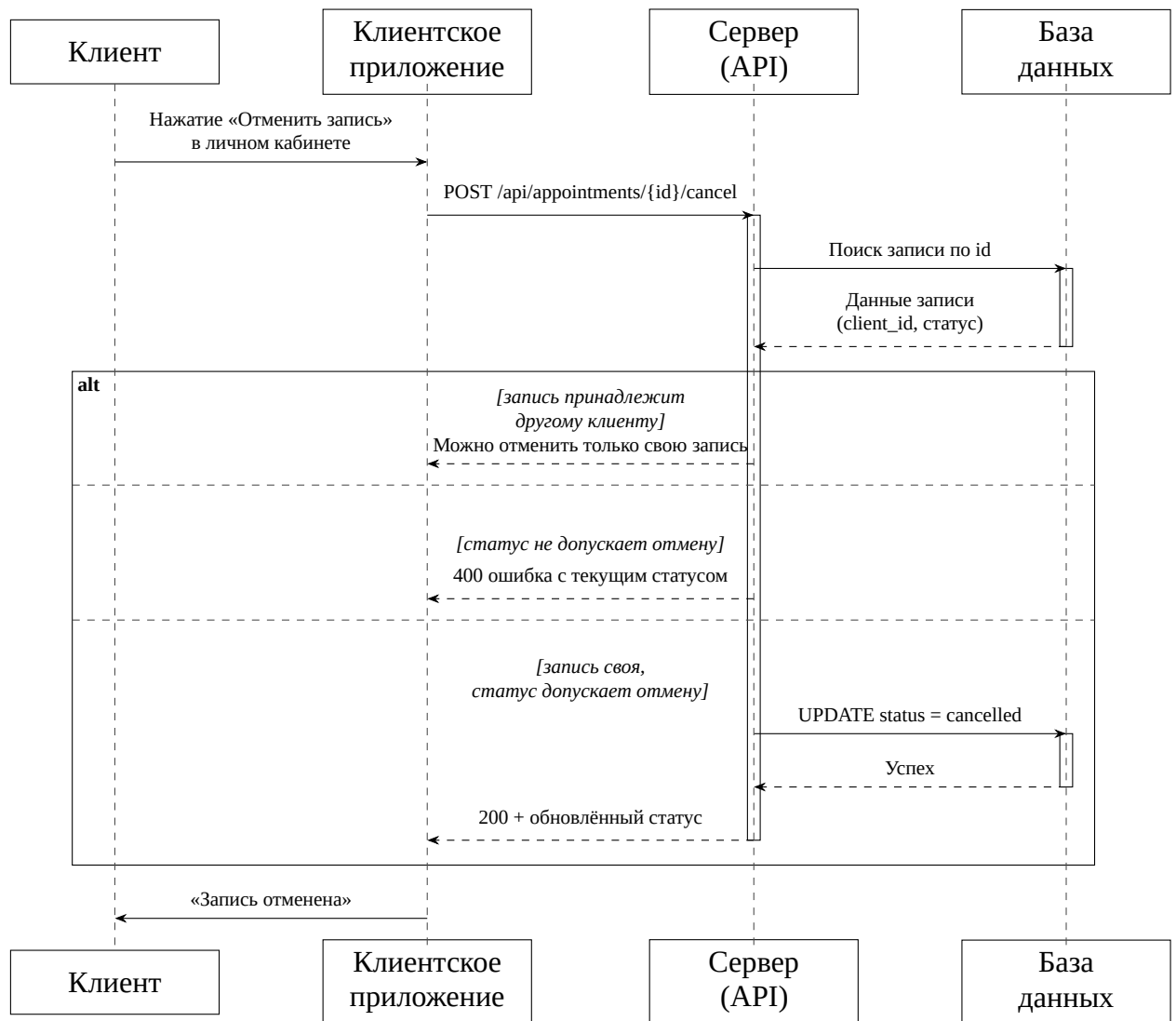


Рисунок А.4 – Диаграмма последовательностей бизнес-процесса «Отмена записи клиентом» (FR-006)

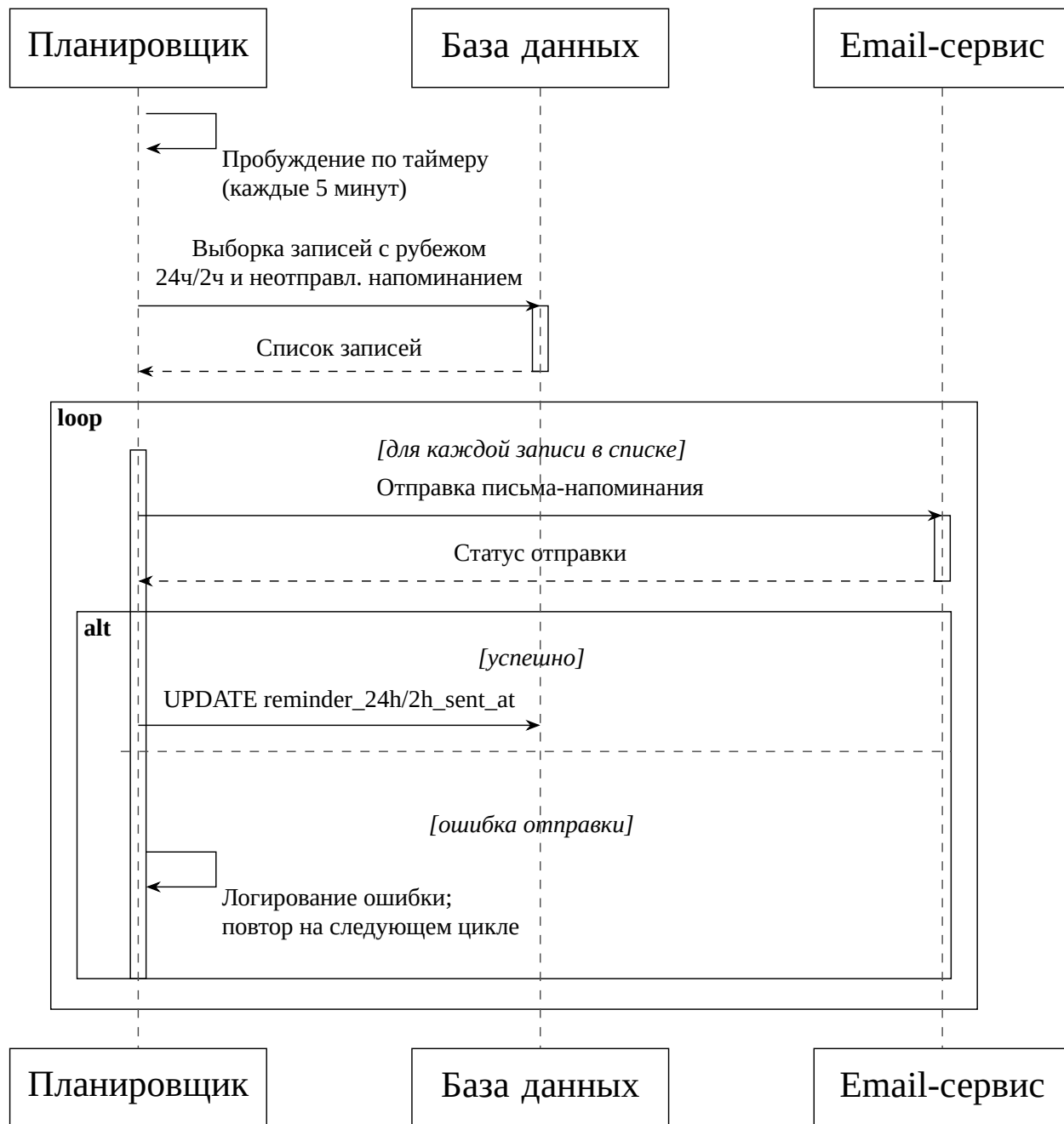


Рисунок А.5 – Диаграмма последовательностей бизнес-процесса «Автоматическая отправка напоминаний» (FR-008)